- **Yaw Correction:** Wheel odometry struggles with heading. A 9-axis IMU provides absolute orientation, allowing the EKF to "snap" the heading back to reality when it drifts.

7.1.2 Part 2: The `robot_localization` Configuration

We use the `ekf_node` from the `robot_localization` package. This node tracks a 15-element state vector:

$$X = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi}, \ddot{x}, \ddot{y}, \ddot{z}]$$

(Position, Orientation, Velocity, Angular Velocity, Acceleration)

File: ***config/ekf.yaml***

```yaml
ekf_filter_node:
  ros__parameters:
    frequency: 30.0
    sensor_timeout: 0.1
    two_d_mode: true  # Racing on a flat track: ignore Z, Roll, and Pitch
    print_diagnostics: true
    publish_tf: true

    # Coordinate Frames
    map_frame: map
    odom_frame: odom
    base_link_frame: base_link
    world_frame: odom

    # 1. Wheel Odometry Input
    odom0: /ackermann_steering_controller/odom
    # Matrix: [x, y, z, roll, pitch, yaw, vx, vy, vz, vroll, vpitch, vyaw, ax, ay, az]
    odom0_config: [true,  true,  false,
                   false, false, false,
                   true,  true,  false,
                   false, false, true,
                   false, false, false]
    odom0_differential: false

    # 2. IMU Input
    imu0: /imu/data
    imu0_config: [false, false, false,
                  false, false, true,
                  false, false, false,
                  false, false, true,
                  true,  false, false]
    imu0_differential: false
    imu0_relative: true
```

7.1.3 Part 3: Implementation (The Launch)

The EKF node must be integrated into the bringup stack alongside the controllers.

```python
from launch_ros.actions import Node

def generate_launch_description():
    # ... previous nodes ...

    ekf_node = Node(
        package='robot_localization',
        executable='ekf_node',
        name='ekf_filter_node',
        output='screen',
        parameters=[os.path.join(get_package_share_directory('
           my_bot'), 'config', 'ekf.yaml')],
        remappings=[('/odometry/filtered', '/odom')] # Remap for
           Nav2 standard
    )

    return LaunchDescription([
        # ...
        ekf_node
    ])
```

7.1.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical State Estimation Pitfalls

- **The "Transform Authority" Nightmare:** A coordinate frame (`odom` → `base_link`) can only have one parent.
  The Pitfall: If the controller and the EKF both publish this transform, the visualizer will flicker violently.
  The Fix: Disable $\texttt{enable}_{odom_t}fin the steering controller and let the EKF be the sole authority.$

- **The "Non-Zero Covariance" Rule:** If a sensor reports zero standard deviation, the EKF treats it as "Infinite Truth." Any glitch can then cause a mathematical singularity (division by zero) and crash the node. Always publish a small, non-zero covariance matrix.
- **Coordinate Frame Alignment:** Ensure the IMU and Encoders agree on the forward (+X) axis. Use static transforms to align `imu_link` perfectly with `base_link` to avoid the filter thinking the car is spinning during straight-line travel.

7.2 Module 7.2: Perception & Mapping (SLAM)

7.2.1 Part 1: The "Why" & The "How" (SLAM vs. EKF)

The "Why"
You might ask: "If we have an EKF, why do we need SLAM?"

- **Local Localization (odom → base_link):** Provided by the EKF. It is high-frequency and smooth but *drifts* over time. After a 500m lap, the EKF might be off by several meters.
- **Global Localization (map → odom):** Provided by SLAM. It uses landmarks (track walls, traffic cones, pillars) to "snap" the car back to its true position on the map, effectively zeroing out the EKF drift.

The "How" (Mapping vs. Localization)

- **Mapping Mode:** The robot explores an unknown track. It uses LiDAR to identify walls, remembers their coordinates, and builds an **Occupancy Grid**—a 2D pixel map where each cell is "Free," "Occupied," or "Unknown."
- **Localization Mode:** The robot is given a pre-saved map. It compares its live LiDAR "view" to the static map. If they match, it knows its exact global coordinates (x, y, θ).

7.2.2 Part 2: Real-world Autonomous Racing Use Cases

- **The Reconnaissance Lap:** Before the race, you drive the car manually (teleop) at low speed. The `slam_toolbox` builds a high-resolution map of the track. You save this as a `.yaml` and `.pgm` file.
- **High-Speed Racing:** During the heat, you switch to "Localization Mode." The car uses the pre-saved map to stay perfectly centered on the racing line, even if wheels are slipping.

7.2.3 Part 3: The `slam_toolbox` Configuration

`slam_toolbox` is the successor to Gmapping. It is faster, more robust, and handles large-scale environments better.

File: *config/mapper_params_online_async.yaml*

```
slam_toolbox:
  ros__parameters:
    # 1. Coordinate Frames (The bridge to our EKF)
    odom_frame: odom
    map_frame: map
    base_frame: base_link
    scan_topic: /scan

    # 2. Mode: Async vs Sync
```

```
    # Async: Best for racing! Drops frames if CPU is busy rather
      than lagging.
    mode: mapping

    # 3. LiDAR Constraints
    max_laser_range: 15.0
    minimum_time_interval: 0.1
    transform_timeout: 0.2

    # 4. SLAM Parameters
    resolution: 0.05            # 5cm per pixel
    max_resample_interval: 0.5

    # Loop Closure (The "Snap" feature)
    do_loop_closing: true
    loop_search_maximum_distance: 3.0
    loop_match_minimum_chain_size: 10
```

7.2.4 Part 4: Implementation (Launch & RViz)

The Launch File Snippet

```
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='slam_toolbox',
            executable='async_slam_toolbox_node',
            name='slam_toolbox',
            output='screen',
            parameters=[os.path.join(get_package_share_directory(
                'my_bot'),
                        'config', 'mapper_params_online_async.
                            yaml'),
                        {'use_sim_time': True}] # Critical for
                            Gazebo!
        )
    ])
```

Visualizing in RViz2

1. Click **Add** → **By Topic** → `/map`.
2. Set the **Reliability Policy** to **Transient Local**. (Crucial: Maps are large and not broadcast constantly; this ensures the node receives the latest "latched" map).

7.2.5 Part 5: Best Practices & Pitfalls (CRITICAL)

Critical Mapping Pitfalls

- **The "Ghosting" Pitfall (LiDAR Vibrations):** If the LiDAR vibrates on a weak bracket, the beams wobble. SLAM perceives moving walls, resulting in a "ghost map" where walls appear doubled.
  The Fix: Use a rigid mount and increase the `minimum_travel_distance` parameter.
- **The "Loop Closure" Jump:** When the car realizes it has seen a corner before, it shifts the entire map to align.
  The Danger: A large jump (e.g., 1m) can jerk the steering of a high-speed controller. Keep the EKF tuned to ensure these jumps remain under 10cm.
- **Sync vs. Async Mode:**
  - **Sync:** Processes every scan. If CPU lags, the whole clock slows. Use only for offline, high-quality map generation.
  - **Async:** Drops scans if the CPU is overwhelmed. **Always use Async for racing.**

7.3 Module 7.3: Nav2 (The Navigation Framework)

7.3.1 Part 1: The "Why" & The "How" (The Modular Navigator)

The "Why"

A race car doesn't just need a path; it needs a strategy. It needs to know how to get from Point A to Point B (**Global Planning**), how to avoid a suddenly appearing obstacle (**Local Control**), and what to do if it gets stuck (**Recovery**).

Nav2 is the industry-standard modular framework for this. It replaces the old ROS 1 Navigation Stack with a more robust, plugin-based architecture designed specifically for production-grade robots.

The "How" (The Three Pillars)

- **Planners (Global):** Computes the long-distance path (the "Line"). It looks at the static map and finds the shortest/fastest route.
- **Controllers (Local):** Also known as the "Follower." It looks at the Global Path and generates the actual velocity and steering commands to follow that path while reacting to live sensor data.
- **Behavior Trees (BT):** The "Orchestrator." It's a logic tree that decides when to plan, when to follow, and when to trigger a "Recovery" behavior (like reversing) if the car is confused.

7.3.2 Part 2: Real-world Autonomous Racing Use Cases

- **Tuning the Inflation Layer:** Walls in a map are single lines of pixels. If a path is planned right next to those pixels, the car's physical width will cause a collision.

We use an **Inflation Layer** to create a "buffer zone." For racing, we tune this to be as tight as possible, allowing the car to "clip the apex" of a turn.

- **Ackermann Dynamics:** Unlike a differential drive robot, a race car has a minimum turning radius. We must use a controller that understands these kinematics, or Nav2 will request physically impossible maneuvers.

7.3.3 Part 3: The Nav2 Configuration (Ackermann Specific)

For racing, we use **Regulated Pure Pursuit (RPP)**. It is designed for high-speed path following and handles the non-holonomic constraints of Ackermann steering perfectly.

File: *config/nav2_params.yaml*

```yaml
controller_server:
  ros__parameters:
    controller_plugins: ["FollowPath"]

    FollowPath:
      plugin: "nav2_regulated_pure_pursuit_controller::
         RegulatedPurePursuitController"
      desired_linear_velocity: 2.0
      lookahead_dist: 0.6       # "Eyes" looking ahead on the
         path
      min_lookahead_dist: 0.3
      max_lookahead_dist: 1.0
      use_velocity_scaled_lookahead_dist: true

      # Ackermann Constraints
      min_approach_linear_velocity: 0.05
      use_approach_vel_scaling: true
      max_allowed_interpolation_dist: 0.1

planner_server:
  ros__parameters:
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner" # Dijkstra/A*
```

7.3.4 Part 4: Costmap Management (Global vs. Local)

- **Global Costmap:**
  Scope: The entire track.
  Purpose: Used by the Planner to find a long-term route. Cares about static obstacles (walls).
- **Local Costmap:**
  Scope: A small "window" (e.g., 3m x 3m) around the car.
  Purpose: Used by the Controller to detect dynamic obstacles (debris, other cars).

7.3.5 Part 5: Best Practices & Pitfalls (CRITICAL)

Critical Navigation Pitfalls

- **The 'Plan vs Control' Frequency Mismatch:** Planning a 50m path is CPU intensive.
  The Pitfall: Running the Global Planner at 20Hz. If it takes 100ms to compute, the steering becomes "jerky."
  Best Practice: Run the Planner at 1-2Hz and the Controller at 20-50Hz.
- **The "Latching" Trap:** Default Behavior Trees might try to "Spin" to recover. An Ackermann car cannot spin.
  The Fix: Replace default BT recovery behaviors with "Wait" or "Replan" logic.
- **Footprint vs. Radius:** Never use `robot_radius` for a rectangular race car.
  Best Practice: Define an explicit **Footprint** (list of $[x, y]$ coordinates for the 4 corners) so Nav2 ensures no corner clips a wall.